

軟體工程

第五課：軟體架構與物件導向設計原則 (Software Architecture & Design)

授課教師：薛念林 教授

資訊工程學系

逢甲大學

第五章：課程藍圖與核心主題

- **5.1 軟體架構基礎** (從需求到設計的關鍵橋樑、微觀與巨觀架構層次)
- **5.2 Kruchten 的 4+1 架構視圖** (邏輯、程序、開發、實體視圖與核心使用案例場景)
- **5.3 經典軟體架構模式** (分層架構、儲存庫、客戶端-伺服器、管線與過濾器、MVC、微服務架構)
- **5.4 架構與系統非功能品質屬性** (效能、安全性、可靠度與可用性之架構權衡)
- **5.5 軟體設計核心根本原則** (四大設計活動、模組化、資訊隱藏、高凝聚度與低耦合度)
- **5.6 SOLID 物件導向設計原則** (SRP 單一職責、OCP 開放封閉、LSP 里氏替換、ISP 介面隔離、DIP 相依反轉)
- **5.7 AI 在架構與軟體設計中的賦能** (AI 副駕駛、自動化重構、設計模式推薦與防範架構盲點)
- **5.8 核心概念統整與參考文獻** (重點填空小測驗與經典學術專著)

本章核心思考問題 (Focus Questions)

- 什麼是軟體架構 (**Software Architecture**)？為什麼它是連結需求與詳細設計的關鍵樞紐？
- 微觀架構 (**Architecture in the Small**) 與 巨觀架構 (**Architecture in the Large**) 有何本質區別？
- Kruchten 的 **4+1** 架構視圖模型包含哪些視角？如何滿足不同利害關係人的多樣化訴求？
- 經典架構模式（分層、儲存庫、主從式、管線過濾器、MVC、微服務）如何組織軟體系統？
- 為什麼高凝聚度 (**High Cohesion**) 與低耦合度 (**Low Coupling**) 是軟體工程不可動搖的黃金法則？
- 物件導向 **SOLID** 五大原則（SRP、OCP、LSP、ISP、DIP）如何預防架構僵化與腐化？
- 生成式 **AI** 能如何有效輔助軟體架構師？人類架構師又該如何守護架構邊界，避免踩入盲點？

5.1 軟體架構基礎

"Software architecture is the set of design decisions which, if made incorrectly, may cause your project to fail."

(軟體架構是一組關鍵的設計決策，這些決策一旦做錯，將可能直接導致你的專案走向徹底失敗。)

— Eoin Woods

”

什麼是軟體架構？(What is Software Architecture?)

軟體系統的根本組織方式，體現於系統各組件之中、組件彼此之間與環境的關係，以及引導系統設計與演進的指導原則。

”

- 關鍵的「橋樑 (Bridge)」角色：
 - 將需求工程 (客戶需要什麼 / What) 銜接至詳細軟體設計 (組件如何建構 / How)。
 - 識別關鍵結構組件，並嚴格定義彼此之間的公開通訊介面契約。
- 架構即核心設計決策：
 - 架構代表專案生命週期早期所做的重大決策，這些決策在後續實作階段變更成本最為高昂。

[SYSTEM_CONTEXT] // DEFINING THE CHASM

Connecting the 'What' to the 'How'



Requirements Engineering (The What)

// Documenting customer needs & specifications

Software Design

Architecture:
Identifying principal structural components and their relationships.

// Defining high-level structure, modules, interfaces & interactions. Ensuring scalability, security, and performance.

Component Design:
Structuring internal algorithms, classes, and logic.

// Detailing low-level logic, data structures, functions, and specific algorithmic implementations.



Implementation (The Code)

// Generating executable software & algorithms

軟體架構的抽象層次：微觀 vs. 巨觀

- **微觀架構 (Architecture in the Small) :**

- 關注單一程式、獨立微服務或用戶端應用程式內部的結構設計。
- 重點在於單一程式如何拆解為子組件、套件 (Packages) 與物件類別。
- 直接決定內部程式碼的可讀性、可測試性與日常重構的迭代速度。

- **巨觀架構 (Architecture in the Large) :**

- 關注跨越多個系統、高度複雜的企業級分散式系統網絡。
- 系統跨越多台伺服器、多個雲端服務區域 (Cloud Regions) 與外部廠商 SaaS 平台。
- 涵蓋大型既有傳統資料庫整合、非同步訊息中介佇列與跨組織 API 整合合約。

將軟體架構明確顯性化的 3 大優勢

1. 促進利害關係人溝通 (Stakeholder Communication)：

- 高階架構圖作為技術團隊、業務主管與外部客戶溝通的共同焦點，無需陷入繁複的程式碼細節即可對焦共識。

2. 早期系統分析與非功能品質驗證 (System Analysis & Verification)：

- 能在寫下第一行程式碼之前，提早分析驗證所提議的設計是否能滿足關鍵的**非功能性需求**（例如高並發低延遲、容錯災難復原、合規稽核）。

3. 大規模架構與資產重用 (Large-Scale Reuse)：

- 經過驗證的架構風格、微服務底座（Chassis）與組件框架，能有系統地在同一領域的多個產品線中大規模重用。

5.2 Kruchten 的 4+1 架構視圖模型

"If you look at an architecture from only one perspective, you are blind to three-quarters of its reality."

(如果你只從單一視角觀察架構，你將對它四分之三的真实面貌視而不見。)

— Philippe Kruchten

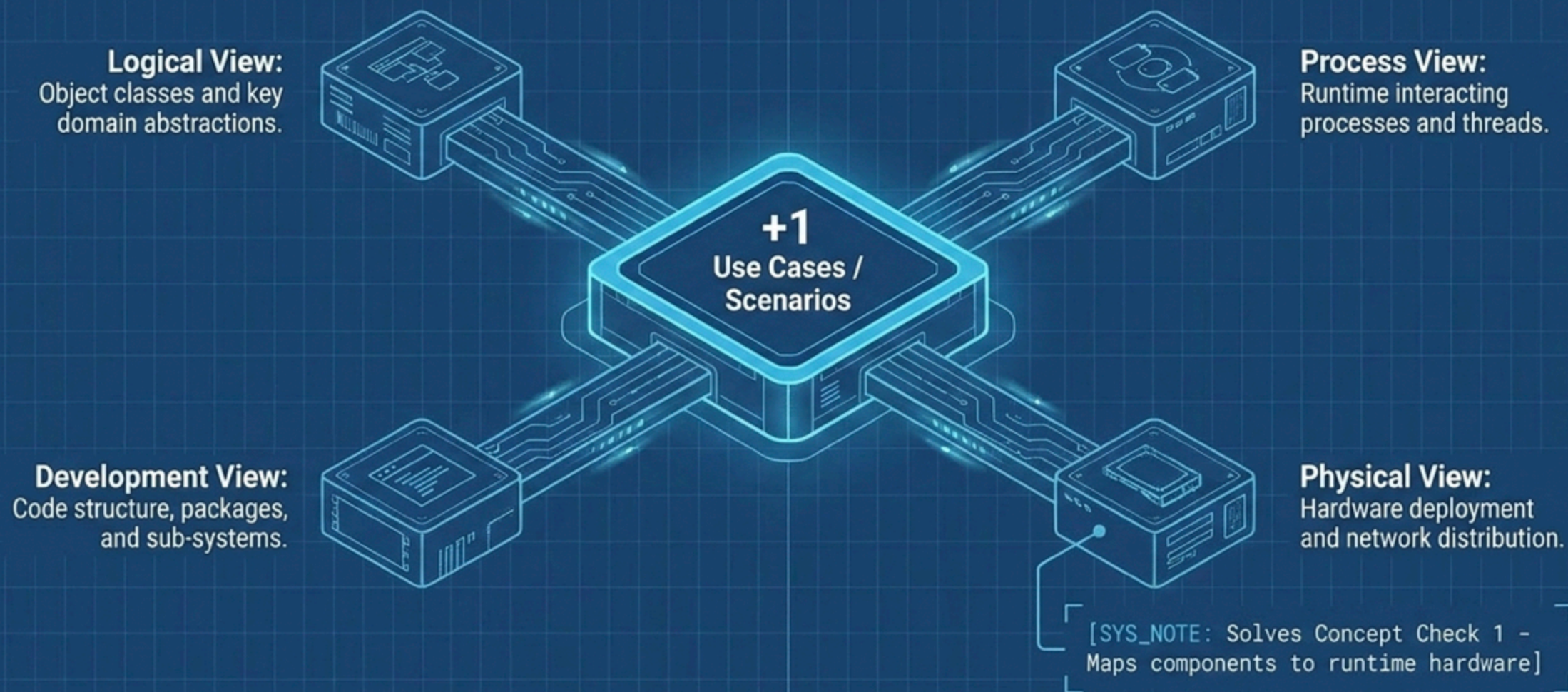
”

Kruchten 的 4+1 架構視圖模型 (4+1 View Model)

- **1. 邏輯視圖 (Logical View / 領域與物件視角) :**
 - 展現核心領域抽象、物件類別、屬性與彼此間的關聯結構 (目標受眾：系統分析師、架構師與客戶)。
- **2. 程序視圖 (Process View / 並發與運行時序視角) :**
 - 展現執行時期程序 (Processes)、執行緒 (Threads)、並發並行機制與跨程序通訊 (目標受眾：系統整合者與效能工程師)。
- **3. 開發視圖 (Development View / 模組與實作視角) :**
 - 展現軟體原始碼模組、套件相依性、建置成品 (Artifacts) 與專案結構 (目標受眾：軟體開發工程師)。
- **4. 實體視圖 (Physical View / 部署與基礎設施視角) :**
 - 展現底層伺服器硬體節點、雲端容器叢集與實體網路拓撲配置 (目標受眾：DevOps 與系統維運管理員)。
- **+1 核心情境 (Scenarios / 使用案例串聯) :**
 - 透過走訪真實的使用者情境旅程，貫穿並驗證上述四大視圖的協同一致性。

[MACRO_LEVEL] // THE 4+1 VIEW MODEL

Five Perspectives, One Unified Blueprint



觀念檢核測驗 1 (CCQ 1)

在 Kruchten 的 4+1 架構視圖模型中，哪一個視圖專門用以展示軟體組件在運行時如何分派部署於實體伺服器硬體、雲端容器叢集與網路拓撲之上？

- **A.** 邏輯視圖 (Logical View)
- **B.** 程序視圖 (Process View)
- **C.** 開發視圖 (Development View)
- **D.** 實體 / 部署視圖 (Physical / Deployment View)



5.3 經典軟體架構模式

"Good architects borrow; great architects reuse proven architectural patterns."

(優秀的架構師懂得借鑒；卓越的架構師則善於重用經過實戰驗證的經典架構模式。)

什麼是軟體架構模式？ (Architectural Patterns)

對良好設計實踐的一種風格化、經過實證檢驗的描述，已在多種不同環境中獲得成功驗證，適用於常見的架構設計情境。

”

- 架構模式的構成要素：

- 模式名稱 (Name)：具備明確意義的專用術語（如 MVC、分層架構、儲存庫模式）。
- 背景與問題 (Context & Problem)：該模式旨在解決的通用結構性挑戰。
- 解決方案 (Solution)：核心組件的職責劃分及其間的通訊互動關係。
- 工程權衡 (Trade-offs)：清晰闡明該模式帶來的架構優勢與伴隨的代價代償。

- 為什麼架構模式如此重要？

- 為跨職能工程團隊建立共同且精確的高階溝通語彙。
- 避免團隊閉門造車，重複發明具備已知缺陷的錯誤結構。

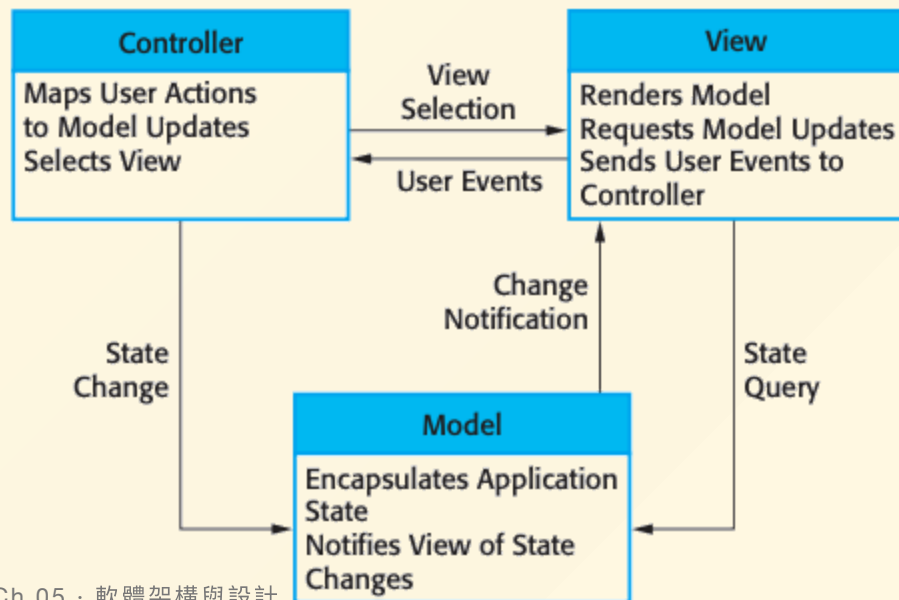
1. 模型-視圖-控制器 (MVC) 模式

- 核心職責分離：

- **模型 (Model)**：管理核心商業領域邏輯、資料狀態與持久化驗證規則。
- **視圖 (View)**：負責使用者介面渲染，將資料視覺化呈現給人類使用者。
- **控制器 (Controller)**：攔截使用者輸入事件（點擊、HTTP 請求），呼叫模型更新，並決定呈現的視圖。

- 核心工程優勢：

- **資料與展示徹底解耦**：多個不同視圖（Web 儀表板、手機原生 App、純 JSON API）可同時觀察同一個底層 Model，無須重複撰寫任何商業邏輯。



2. 分層架構模式 (Layered Architecture)

- 堆疊式抽象層設計：
 - 將系統功能組織為垂直有序的層級。每一層僅為其正上方的相鄰層級提供服務，且僅依賴其正下方的底層服務。
- 現代企業級標準四層架構：
 - 展示層 (**Presentation Layer**)：使用者介面、REST API 端點、視圖控制器。
 - 應用 / 服務層 (**Service Layer**)：業務工作流程編排、跨領域交易事務邊界。
 - 領域 / 業務層 (**Domain Layer**)：純粹的領域商業規則、實體與領域演算法。
 - 基礎設施 / 資料層 (**Infrastructure Layer**)：資料庫讀寫存取、外部第三方服務整合。
- 關鍵架構價值：
 - 隔離變更與高可替換性：只要介面合約維持不變，更換底層實作（例如將 Oracle 換成 PostgreSQL）完全不會波及上層商業領域邏輯。

3. 儲存庫架構模式 (Repository Architecture)

- 集中式資料共享機制：

- 各個獨立子系統透過一個共用的中央儲存庫 (Repository / Data Store) 交換大規模、高關聯的複雜資料。
- 各子系統獨立運作，彼此不直接溝通；一切互動均嚴格透過對中央儲存庫的狀態讀寫來達成。

- 業界經典應用場景：

- 現代整合開發環境 (IDE，如 VS Code 語法伺服器、Eclipse 工作區 AST 儲存庫)。
- 電腦輔助設計 (CAD) 大型工程繪圖套件。
- 醫院電子病歷 (EHR) 中央病患核心資料庫。

- 架構優劣權衡：

- 優勢： 高效共享海量、高維度的互聯資料集，避免在處理程序間頻繁傳輸大量資料。
- 劣勢： 中央儲存庫極易淪為系統的單點故障 (**Single Point of Failure, SPOF**) 與效能瓶頸。

4. 主從式架構與管線過濾器模式

- 客戶端-伺服器架構 (Client-Server Architecture) :

- 系統功能分派由提供專屬服務（運算、資料儲存、身分認證）的伺服器端 (Server)，與透過網路發出請求的客戶端 (Client) 共同組成。
- 權衡：易於在通用硬體上水平擴展，但本質上受制於網路延遲與連線飽和度瓶頸。

- 管線與過濾器架構 (Pipe-and-Filter Architecture) :

- 資料處理被組織為一系列獨立的轉換步驟 (Filters / 過濾器)，彼此透過資料串流 (Pipes / 管線) 相互串接。
- 第 N 個過濾器的輸出，直接作為第 $N + 1$ 個過濾器的輸入串流。
- 經典範例：Unix 終端機管線指令 (`cat log.txt | grep 'ERROR' | sort | uniq -c`)。
- 最佳適用情境：批次帳單結算、大數據分析管線、編譯器語法解析。

5. 微服務架構 (Microservices, MSA) vs. 單體架構

- **單體架構 (Monolithic Architecture) :**

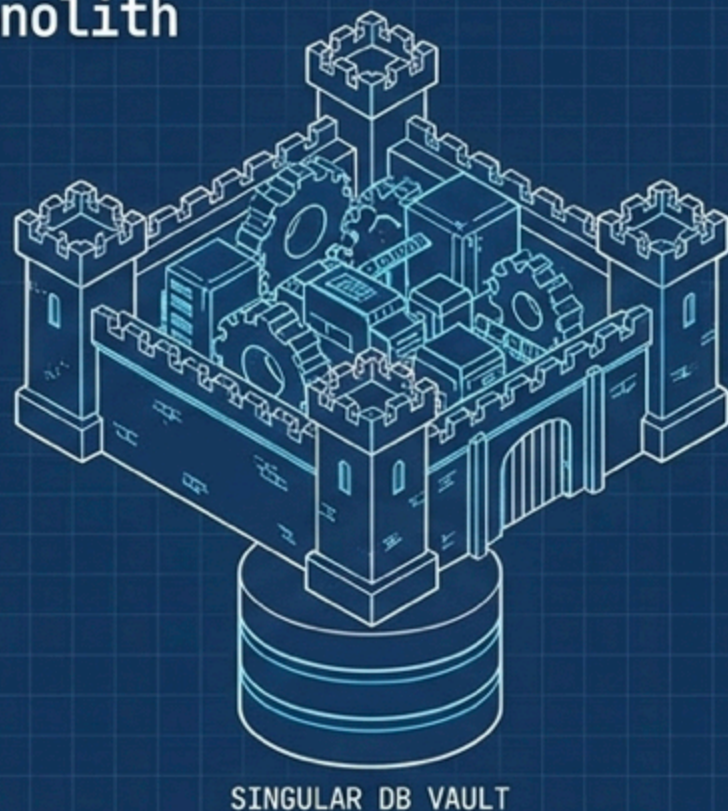
- 所有的系統功能、業務邏輯與資料庫存取，皆打包並部署為單一、不可分割的可執行二進位檔。
- **優點：** 初期開發極速、除錯直觀、零跨網路序列化額外開銷。
- **痛點：** 擴展時必須複製整個應用；單一模組 Memory Leak 或崩潰將拖垮全系統。

- **微服務架構 (Microservices Architecture) :**

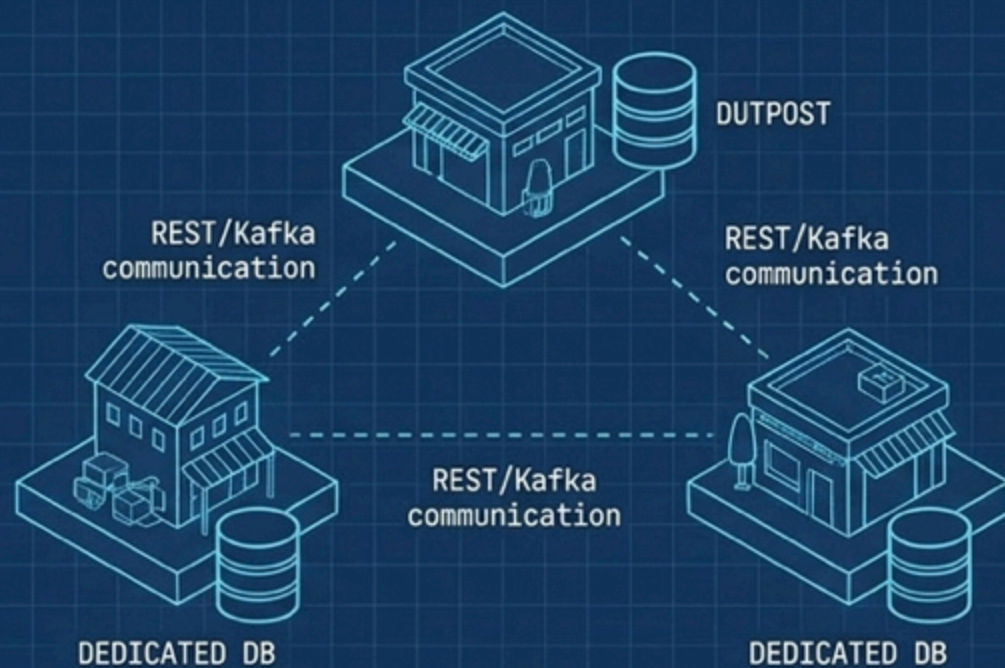
- 圍繞著業務領域能力 (**Business Capabilities**)，將系統拆解為一套由微小、可獨立部署的服務所組成的叢集。
- 每個微服務擁有自己完全獨立的私有資料庫（去中心化資料管理）。
- 服務間透過輕量級 REST/gRPC 或非同步訊息中介（Kafka / RabbitMQ）進行通訊。

Monolithic Fortresses vs. Microservice Networks

The Monolith



Microservices Architecture (MSA)



Simplicity

Resilience & Independent Scaling

觀念檢核測驗 2 (CCQ 2)

哪一個經典架構模式明確將使用者介面呈現與畫面佈局，從底層商業領域資料管理與使用者輸入事件處理中徹底解耦分離？

- **A.** 管線與過濾器模式 (Pipe and Filter Pattern)
- **B.** 模型-視圖-控制器模式 (Model-View-Controller, MVC)
- **C.** 儲存庫模式 (Repository Pattern)
- **D.** 主從式架構模式 (Client-Server Pattern)



5.4 架構與系統品質屬性

"Architecture is where requirements meet reality through trade-off analysis."

(架構，是軟體需求透過權衡分析與工程現實交會的所在。)

”

軟體架構直接決定非功能品質屬性

- **效能表現 (Performance) :**
 - **架構手段：** 將關鍵核心計算封裝於高凝聚組件中，盡量減少跨行程與跨網路的遠端呼叫；引入多層級分散式快取（如 Redis / Memcached）。
- **資訊安全性 (Security) :**
 - **架構手段：** 採用縱深防禦（Defense-in-Depth）多層安全架構；將高價值資產置於內部受保護核心層；外圍由 API 閘道強制執行身分認證。
- **系統安全性 (Safety，如航空/醫療) :**
 - **架構手段：** 將關乎生命安全的關鍵邏輯嚴格隔離於獨立、具備硬體冗餘備援的子系統，杜絕故障擴散。
- **高可用性 (Availability) :**
 - **架構手段：** 全面消除單點故障（SPOF）；部署叢集冗餘實例、自動化健康檢查心跳與跨區域主動容錯移轉（Active-Active Failover）。
- **可維護性 (Maintainability) :**
 - **架構手段：** 拆解為細粒度、低耦合的模組，組件間一律嚴格透過穩定的抽象介面溝通。

Zoom Levels

Architecture in the Large:
Complex enterprise systems,
multi-system distributed networks.

Architecture in the Small:
Individual programs,
decomposition into
sub-components.

Performance
Driven by localized operations
and minimized communication.

Security
Driven by layered architectures
protecting inner data assets.

Safety
Driven by isolating critical features
in dedicated sub-systems.

Availability
Driven by redundant components
and automated failover.

Maintainability
Driven by fine-grained, loosely
coupled components.

Driven by localized operations
and minimized communication.

Driven by layered architectures protecting inner data assets.

Driven by isolating critical features in dedicated sub-systems.

Driven by redundant components and automated failover.

Driven by fine-grained, loosely coupled components.

5.5 軟體設計核心原則

"There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies."

(建構軟體設計有兩種方式：一種是讓它簡單到明顯看不出任何缺陷；另一種則是讓它複雜到看不出任何明顯的缺陷。)

— C. A. R. Hoare (圖靈獎得主)

”

軟體設計的 4 大基本活動 (4 Design Activities)

1. 架構設計 (Architectural Design / 巨觀結構)：

- 識別關鍵子系統、定義服務邊界，以及設計模組間的主要通訊管線。

2. 介面設計 (Interface Design / 契約規範)：

- 明確定義各組件之間公開、精確且無歧義的契約介面（REST API 規範、介面方法簽章）。

3. 組件與詳細設計 (Component Design / 微觀實作)：

- 深入設計各類別內部的資料結構、核心演算法與私有輔助方法邏輯。

4. 資料庫與資料結構設計 (Data Persistence Design / 持久化)：

- 設計關聯式實體關聯圖（ERD）、文件集合（Collections）綱要或記憶體資料結構。

經典設計準則：模組化、抽象化與關注點分離

- **1. 抽象化 (Abstraction) :**
 - 將低階繁複的實作細節隱藏在清晰的高階概念介面之後。讓工程師在思考複雜系統時不至於大腦認知超載。
- **2. 封裝與資訊隱藏 (Encapsulation & Information Hiding) :**
 - *David Parnas 原則*：將狀態資料與專屬操作緊密綁定於模組內部，並將私有變數徹底隱藏。杜絕外部程式碼非法修改內部狀態所帶來的災難性副作用。
- **3. 關注點分離 (Separation of Concerns, SoC) :**
 - 將軟體拆解為互不重疊的獨立功能面向（例如：徹底分離 UI 渲染排版邏輯與資料庫 SQL 查詢）。
- **4. 模組化 (Modularity) :**
 - 將系統劃分為內聚完整、可獨立測試且能隨時替換抽換的獨立單元。

至高黃金法則：高凝聚度與低耦合度

- **凝聚度 (Cohesion)：**

- 衡量單一類別或模組內部職責彼此關聯的緊密與專注程度。
- **軟體工程目標：高凝聚度 (High Cohesion)**（一個類別應該只專注做好一件事，並且把它做到極致）。

- **耦合度 (Coupling)：**

- 衡量不同獨立模組之間直接相互依賴的強烈程度。
- **軟體工程目標：低耦合度 (Low Coupling)**（模組之間透過抽象介面溝通，對彼此內部實作細節一無所知）。

- **不可動搖的通用黃金法則：**

最大化凝聚度，最小化耦合度！(Maximize Cohesion, Minimize Coupling!)

高凝聚讓程式碼極易理解與維護；低耦合讓單一模組的修改絕不會在全系統引發連鎖骨牌效應。

”

凝聚度與耦合度：不良設計 vs. 優良設計對比

//  BAD: 低凝聚度、高耦合度的反面教材 (萬能上帝類別 "God Class")

```
class OrderManager {  
    public void processOrder() {  
        // 直接編寫並執行 MySQL 原生 SQL 字串語法  
        // 混雜複雜的 VIP 會員折扣商業數學計算  
        // 拼接組裝訂單發票的純 HTML 字串  
        // 建立低階 TCP/SMTP Socket 網路連線發送電子郵件  
    }  
}
```

//  GOOD: 高凝聚度、低耦合度的優良設計 (解耦的單一職責協同類別)

```
class OrderProcessor {  
    private final DiscountCalculator discountCalculator;  
    private final OrderRepository orderRepository;  
    private final NotificationService notificationService;  
  
    public OrderProcessor(DiscountCalculator dc, OrderRepository repo, NotificationService ns) {  
        this.discountCalculator = dc;  
        this.orderRepository = repo;  
        this.notificationService = ns;  
    }  
}
```

觀念檢核測驗 3 (CCQ 3)

在軟體系統設計中，將高凝聚度 (High Cohesion) 與 低耦合度 (Low Coupling) 兩大工程原則結合應用，所帶來最核心的實務效益為何？

- **A.** 透過將所有函式內聯整合至單一二進位檔案，最大化 CPU 執行時的運算效能。
- **B.** 打造出高度模組化的程式碼架構，使得單一類別的規格修改對其他模組產生極小連鎖影響。
- **C.** 強制規範系統內部所有的核心資料結構，必須全部集中儲存於全域共享變數空間中。
- **D.** 徹底消除生產環境持續交付流程中，編寫自動化整合測試與回歸測試的必要性。



5.6 SOLID 物件導向設計原則

"Clean code always looks like it was written by someone who cares."

(乾淨的程式碼讀起來，永遠就像是由深具責任心與熱忱的人所撰寫。)

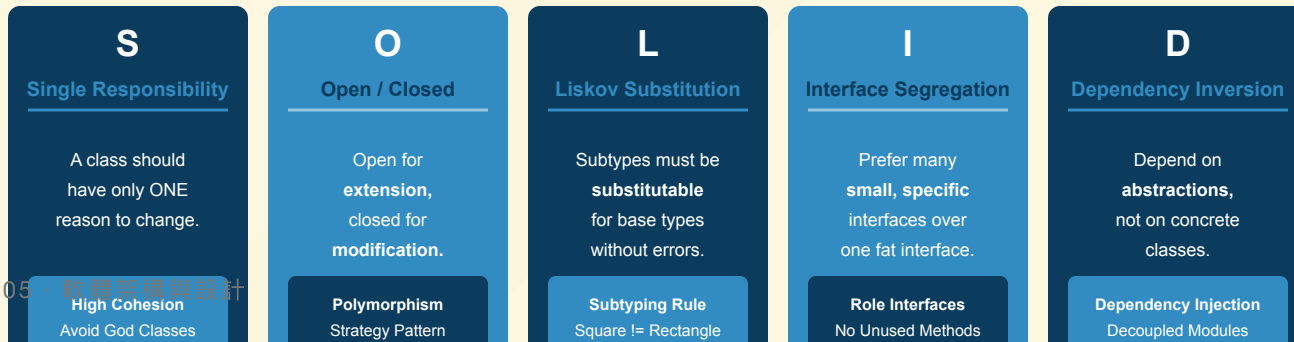
— Robert C. Martin (*Uncle Bob*)

”

概覽：SOLID 五大設計原則

- **S – 單一職責原則 (Single Responsibility Principle, SRP) :**
 - 一個類別應該有且僅有一個引起它改變的原因。
- **O – 開放封閉原則 (Open / Closed Principle, OCP) :**
 - 軟體實體應對擴充開放，但對修改封閉。
- **L – 里氏替換原則 (Liskov Substitution Principle, LSP) :**
 - 子型別必須能夠替換掉其基礎父型別，且不破壞程式的正確性。
- **I – 介面隔離原則 (Interface Segregation Principle, ISP) :**
 - 用戶端不應被迫依賴其不需要的肥大介面方法。
- **D – 相依反轉原則 (Dependency Inversion Principle, DIP) :**
 - 高階模組不應依賴低階模組；兩者皆應依賴抽象介面。

The SOLID Object-Oriented Design Principles



S – 單一職責原則 (Single Responsibility Principle, SRP)

- 核心原則：

一個類別應該有且僅有一個引起它改變的原因。

”

- 為什麼 SRP 至關重要？

- 當一個類別承擔多重職責時（例如資料庫存取 + PDF 報表渲染 + 電子郵件寄送），行銷團隊要求更換 Email 格式的修改，極易在無意間引發資料庫儲存邏輯的致命錯誤！

//  違反 SRP：擁有 3 個截然不同的改變理由（資料庫存取、PDF 渲染、SMTP 郵件協定）

```
class UserRepositoryManager {  
    public void fetchUserData() { /* 執行資料庫 SQL 查詢 */ }  
    public void generatePdfReport() { /* 複雜的 PDF 排版繪圖引擎 */ }  
    public void sendEmailReport() { /* SMTP 網路協定傳輸郵件 */ }  
}
```

//  遵循 SRP：拆解為 3 個職責專一、各司其職的獨立類別

```
class UserRepository { public User fetchUserData() { ... } }  
class PdfReportFormatter { public byte[] generatePdf(User u) { ... } }  
class EmailService { public void sendEmail(String to, byte[] data) { ... } }
```

O – 開放封閉原則 (Open / Closed Principle, OCP)

- 核心原則：

軟體實體（類別、模組、函式）應當對擴充開放，但對修改封閉。

”

- 關鍵工程實踐機制：

- 善用介面與多型 (**Polymorphism**)（例如策略模式 Strategy Pattern），徹底取代脆弱、層層巢狀且頻繁修改的 `if-else` 或 `switch` 型別檢查判斷。

// ✗ 違反 OCP：每當行銷部門推出全新折扣活動，就必須被迫修改既有已測試的程式碼！

```
class DiscountCalculator {
    public double calculate(String tier, double price) {
        if (tier.equals("VIP")) return price * 0.8;
        else if (tier.equals("STUDENT")) return price * 0.9;
        return price; // 每次新增促銷活動，都必須冒險修改這個核心方法！
    }
}
```

// ✓ 遵循 OCP：透過新增實作介面的全新類別來完成擴充，零修改既有程式碼！

```
interface DiscountStrategy { double apply(double price); }
class VipDiscount implements DiscountStrategy { public double apply(double p) { return p * 0.8; } }
class StudentDiscount implements DiscountStrategy { public double apply(double p) { return p * 0.9; } }
```

L – 里氏替換原則 (Liskov Substitution Principle, LSP)

- 核心原則：

子型別必須能夠在程式中無縫替換掉其父型別，且不影響任何既有程式執行的正確性。

”

- 經典反面教材：正方形繼承長方形 (Square extends Rectangle)：

- 在幾何學上，正方形是長方形的一種。但在物件導向程式碼中，修改正方形的寬度必須連帶強制改變高度，徹底打破了長方形呼叫端既有的不變量假設！

// **✗ 違反 LSP**: Square 破壞了調用端對 Rectangle 不變量行為的合理期待！

```
class Rectangle {  
    protected int width, height;  
    public void setWidth(int w) { this.width = w; }  
    public void setHeight(int h) { this.height = h; }  
    public int getArea() { return width * height; }  
}  
  
class Square extends Rectangle {  
    @Override public void setWidth(int w) { this.width = w; this.height = w; }  
    @Override public void setHeight(int h) { this.width = h; this.height = h; }  
}
```

// 呼叫端執行: rect.setWidth(5); rect.setHeight(4); 期待面積為 20，若傳入 Square 卻得到 16！

I – 介面隔離原則 (Interface Segregation Principle, ISP)

- 核心原則：

用戶端不應該被迫依賴它所不需要使用的介面方法。

”

- 「肥大介面 (Fat Interface)」所帶來的危害：

- 避免設計包山包海、動輒包含數十個方法的龐大介面。應積極拆解為小巧、專精且針對特定角色的專用介面。

// **✗** 違反 ISP：「肥大介面」迫使陽春印表機必須去實作它根本不具備的掃描與傳真功能！

```
interface MultifunctionDevice {  
    void print();  
    void scan();  
    void fax();  
}  
  
class SimplePrinter implements MultifunctionDevice {  
    public void print() { /* 正常列印 */ }  
    public void scan() { throw new UnsupportedOperationException(); } // 經典反模式！  
    public void fax() { throw new UnsupportedOperationException(); }  
}
```

D – 相依反轉原則 (Dependency Inversion Principle, DIP)

- 核心原則：

高階模組不應依賴低階模組，兩者皆應依賴抽象介面；抽象不應依賴細節，細節應依賴抽象。

”

- 關鍵工程實踐：相依性注入 (Dependency Injection, DI)：

- 高階商業邏輯服務絕不可以在內部直接以 `new MySQLDatabase()` 寫死具體資料庫！應該透過建構子注入抽象的 `Database` 介面。

//  違反 DIP：高階商業邏輯直接硬編碼寫死綁定具體的 `MySQL` 實作！無法抽換與測試！

```
class OrderService {
    private MySQLDatabase db = new MySQLDatabase(); // 緊密耦合！單元測試時無法替換！
    public void completeOrder(Order o) { db.insert(o); }
}
```

//  遵循 DIP：高低階雙方均依賴抽象介面；透過建構子落實相依性注入 (DI)

```
interface Database { void insert(Order o); }
```

```
class OrderService {
    private final Database db; // 嚴格只依賴抽象介面！
```

SOLID: The Good vs. Bad Matrix

Principle	✗ The Violation	✓ The Fix	Core Benefit
SRP	Changes to PDF layout and Email server break the same class.	Separate PDF and Email into distinct classes.	Isolates requirement changes.
OCP	Using massive if-else or switch blocks on object types.	Use Polymorphism and interfaces (Strategy Pattern).	Add features without editing tested code.
LSP	Subclasses throwing unexpected exceptions breaking base assumptions.	Derived classes strictly fulfill base invariants.	Prevents polymorphic runtime crashes.
ISP	Dummy method implementations throwing <code>UnsupportedOperationException</code> .	Segregate into multiple small, focused interfaces.	Prevents forced dependency on unused methods.
DIP	High-level business logic instantiates concrete database engines.	Use Dependency Injection (DI) and depend on interfaces.	Allows swapping services without rewriting logic.

觀念檢核測驗 4 (CCQ 4)

哪一個 SOLID 物件導向設計原則明確主張：當需要為系統引入新功能時，應當透過「新增新的類別」來擴充，而非直接修改既有已通過測試的原始碼？

- **A.** 單一職責原則 (Single Responsibility Principle, SRP)
- **B.** 開放封閉原則 (Open / Closed Principle, OCP)
- **C.** 介面隔離原則 (Interface Segregation Principle, ISP)
- **D.** 里氏替換原則 (Liskov Substitution Principle, LSP)



5.7 AI 在架構與軟體設計中的賦能與盲點

"AI can recommend design patterns and refactor code smells, but human architects must steer the system's destiny."

(AI 能夠推薦設計模式並重構不良程式碼氣味，但唯有身為人類的架構師，才能真正掌舵系統的終極命運。)

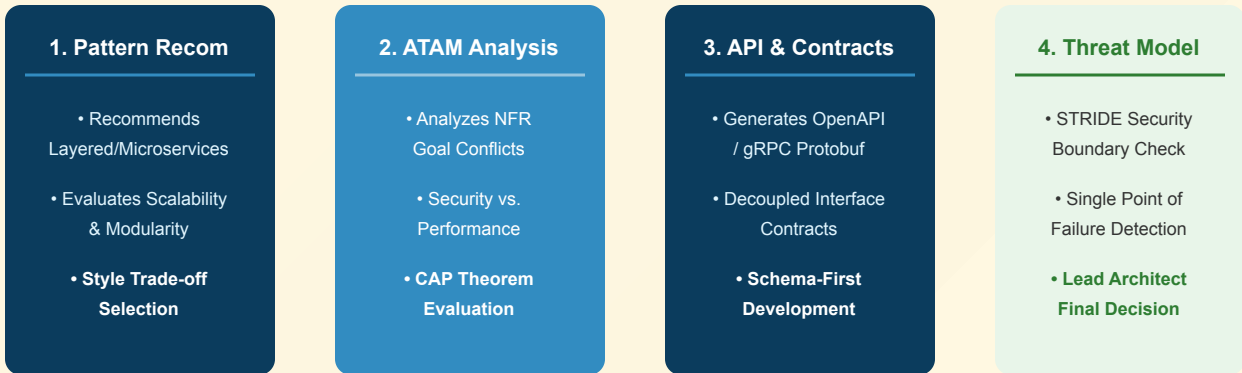
”

AI 在架構與設計中的角色：副駕駛典範 (The Copilot Paradigm)

- 分析型架構副駕駛的跨越：

- LLM 能深入解析複雜系統規格，針對特定場景推薦適配的架構模式（如微服務 vs. 事件驅動架構）。
- 能在數秒內極速產出系統間通訊的介面契約（OpenAPI / Swagger 規格書、gRPC `.proto` 檔案與資料庫 DDL）。

How AI Assists Architectural Design (Architect Copilot)



- 核心架構價值：大幅加速跨方案權衡分析（Trade-off Analysis），並自動化生成子系統間純淨無瑕的標準化介面契約樣板。

AI 在軟體設計中的 4 大核心落地應用

- **1. 自動化 SOLID 重構建議 (Automated SOLID Refactoring) :**
 - 掃描分析老舊巨石程式庫，精準揪出違反 SRP 的萬能上帝類別，並主動提議套用策略模式或工廠模式以落實 OCP。
- **2. 程式碼不良氣味與耦合度偵測 (Code Smell & Coupling Detection) :**
 - 跨模組靜態偵測高度依賴、循環參照 (Circular Dependencies)、重複程式碼與「散彈式修改 (Shotgun Surgery)」熱點。
- **3. 經典設計模式精準推薦 (Design Pattern Recommendation) :**
 - 針對反覆發生的架構摩擦阻力，智慧推薦最契合的 GoF 設計模式（如觀察者 Observer、裝飾者 Decorator、轉接器 Adapter、建造者 Builder）。
- **4. TDD 測試導向設計之介面與 Mock 模擬自動生成 :**
 - 自動為具體業務邏輯抽離解耦介面，並生成 Mock 測試替身物件，極速推進測試驅動開發流程。

人機協同防護：防範 AI 架構盲點 (Architectural Blind Spots)

- 未經驗證而盲目採納 AI 架構建議的潛在風險：
 - 過早引入分散式複雜度 (**Premature Distributed Complexity**)：針對三個工程師兩週就能以單體架構搞定的單純內部工具，AI 卻動輒推薦微服務、Kubernetes 與 Kafka 叢集。
 - 組織脈絡盲目 (**Context Blindness**)：AI 完全不知道你們團隊的真實技術棧深度、雲端預算上限，或是企業特有的資安合規紅線。
 - 表面性無效重構 (**Superficial Refactoring**)：僅僅更換變數命名或套用空洞外殼，未真正解決深層的演算法與架構耦合。
- 軟體架構至高黃金守則：

「**AI** 負責提出權衡分析方案；主架構師全權承擔最終決策！」

(AI Proposes Trade-offs; Lead Architects Make Decisions!)

人類工程師必須深入評估團隊組織脈絡，為系統的長治久安承擔最終責任。

”

5.8 核心複習與參考文獻

"Architecture is the decisions that you wish you could get right the first time."

(架構，就是那些你希望自己在第一次做的時候就能做對的關鍵決策。)

”

核心觀念統整：互動填空小測驗

測試你對本章核心概念的掌握度：

1. 架構將系統組織為垂直有序的層級，每一層僅為其正上方的相鄰層級提供服務。
2. 在 模式中，多個獨立的子系統完全圍繞並透過一個共用的中央資料庫進行資訊交換。
3. Kruchten 的 視圖模型結合了邏輯、程序、開發與實體視圖，並以使用案例情境為串聯核心。
4. 衡量單一模組內部的專注程度，而 則衡量不同模組之間的依賴緊密程度。
5. 原則主張軟體實體應該對擴充開放，但對修改封閉。
6. 原則規定高階商業邏輯絕不應直接依賴低階模組，兩者皆應依賴抽象介面。

經典文獻與延伸閱讀 (References)

- 奠基經典教科書與權威文獻：
 - Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 6 (Architecture) & Chapter 7 (Design). Pearson.
 - Kruchten, P. (1995). "Architectural Blueprints — The '4+1' View Model of Software Architecture." *IEEE Software*, 12(6), 42-50.
 - Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
 - Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- 現代分散式系統與微服務架構指引：
 - Fowler, M. (2014). *Microservices: A definition of this new architectural term*. martinfowler.com
 - Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture*. O'Reilly Media.